

SOFTWARE REQUIREMENTS SPECIFICATION

Mobile Shop Manager Pro

Offline Desktop-Based Mobile Shop Management Software

Stack: Tauri · Rust · SQLite

Version 1.0 | August 2026

Prepared by: Noor Ullah

Classification: Confidential

1. Introduction

1.1 Purpose

Mobile Shop Manager Pro is an offline desktop application designed to help mobile shop owners manage their complete business operations digitally without requiring internet connectivity.

The system is built on a modern Tauri + Rust + SQLite stack — delivering a small, fast, cross-platform installer with native-grade performance and encrypted local data storage.

1.2 Document Scope

This SRS covers all functional requirements, module specifications, the Tauri/Rust architecture, security measures, database design, and build roadmap for version 1.0.

1.3 Definitions & Abbreviations

Term	Definition
SRS	Software Requirements Specification
Tauri	Open-source framework for building desktop apps with Rust backend and web frontend
Rust	Systems programming language — memory-safe, high performance, no garbage collector
IPC	Inter-Process Communication — how the frontend (WebView) calls Rust backend functions
WebView2	Microsoft's Chromium-based rendering engine used by Tauri on Windows
MVVM	Model–View–ViewModel — UI architecture for the React frontend
ORM	Object-Relational Mapper — maps Rust structs to database tables
SQLite	Embedded, serverless, file-based relational database — zero configuration
SQLCipher	Open-source encrypted extension for SQLite (AES-256 full-database encryption)
Diesel	Safe, compile-time checked ORM for Rust
Tauri Command	A Rust function exposed to the frontend via <code>#[tauri::command]</code> — the IPC bridge
AES-256	Advanced Encryption Standard, 256-bit key — industry encryption standard
IMEI	International Mobile Equipment Identity — unique 15-digit device identifier

2. Project Objectives

- Provide a fast, professional offline desktop app for mobile shop owners
- Replace manual registers and Excel-based management
- Maintain accurate real-time inventory for mobiles and accessories
- Track daily sales, payments, and profits automatically
- Manage customer credit and installment payments efficiently
- Generate professional invoices with IMEI and warranty details
- Protect all business data with AES-256 database encryption
- Support multiple staff roles with fine-grained permissions
- Deliver a commercial product with hardware-bound offline license activation
- Provide an AES-256 encrypted backup and restore system

3. Target Users

Role	Responsibilities	Access Level
Shop Owner / Admin	Full business management, reports, settings, user control	Full Access
Sales Staff	Create sales, add customers, receive payments, view own records	Limited
Accountant	View financial records, manage expenses, generate reports	Read + Finance

4. System Scope

4.1 Included in Version 1.0

- ✓ Mobile and accessories inventory management
- ✓ Sales, billing, and invoice generation
- ✓ Customer management and purchase history
- ✓ Credit and dues tracking with installment records
- ✓ Supplier management and purchase recording
- ✓ Expense tracking and categorization
- ✓ Profit, sales, and inventory reports
- ✓ Multi-user with role-based access control
- ✓ Hardware-bound offline license activation system
- ✓ AES-256 encrypted backup and restore
- ✓ Admin settings and customization

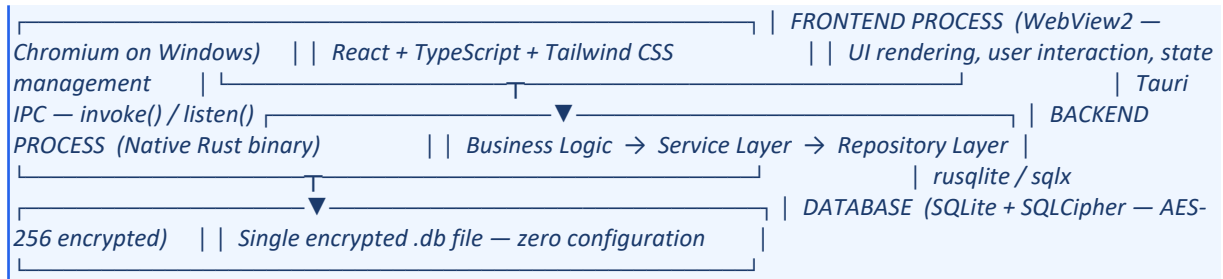
4.2 Excluded from Version 1.0

- ✗ Online marketplace or e-commerce
- ✗ Repair / service job management
- ✗ Barcode or QR scanning hardware
- ✗ Cloud synchronization
- ✗ Online payment gateway integration

5. System Architecture

5.1 Tauri Application Model

Tauri applications consist of two processes that communicate via an IPC bridge:



5.2 How Tauri IPC Works

The frontend (React) cannot directly access the database or filesystem. Every database operation is requested through a Tauri Command — a Rust function tagged with `#[tauri::command]`. This enforces security and business logic at the Rust layer.

```
// Frontend — React calls a Rust function const result = await invoke("create_sale", { saleData: payload }); //  
Backend — Rust handles the command #[tauri::command] fn create_sale(sale_data: SaleDto, state: State<AppState>) -> Result<SaleResult, AppError> { sale_service::create(sale_data, &state.db) }
```

5.3 Clean Architecture Layers

Layer	Technology	Responsibility
UI / View	React + TypeScript + Tailwind	Renders screens, handles user input, shows data
State Mgmt	Zustand (React)	Frontend state, caching, UI-level logic
IPC Bridge	Tauri Commands + Events	Secure channel between WebView and Rust
Service Layer	Rust modules (sale_service.rs, stock_service.rs)	Business rules, validation, calculations
Repository	Rust structs over rusqlite/sqlx	All SQL queries — abstracted from services
Database	SQLite + SQLCipher	Encrypted local file-based data storage

6. System Modules

6.1 User Authentication Module

Feature	Detail
Login	Username + password — password hashed with Argon2id in Rust (more secure than bcrypt)
Session Token	On login, Rust generates a signed JWT stored in memory — never on disk
Role Management	Admin, Employee, Accountant — roles stored in DB, loaded on login
Permission Check	Every Tauri command validates the session token and role before executing
Auto Logout	Frontend timer — session expires after configurable idle period
Activity Log	Every login, sale, edit, and delete recorded with user ID and timestamp in Rust

6.2 Dashboard Module

- Today's total sales amount and transaction count
- Today's net profit (revenue minus purchase cost)
- Total products in inventory with low-stock alerts
- Customer pending payment summary
- Supplier outstanding payment summary
- Recent 10 transactions with quick navigation links
- Monthly sales trend chart (Recharts library — React)

6.3 Product Management Module

6.3.1 Mobile Phones

Field	Type	Validation (Rust-side)
Brand	Dropdown	Required — validated against brands table
Model	Text	Required — max 100 characters
RAM / Storage	Dropdown	Required — configurable variants
Color	Text	Required
PTA Status	Dropdown	Approved / Non-PTA / Grey
IMEI	Text	Required — Luhn algorithm check in Rust — unique constraint in DB
Purchase Price	Decimal	Required — positive f64
Sale Price	Decimal	Required — warning returned if less than purchase price

Field	Type	Validation (Rust-side)
Supplier	FK	Required — links to suppliers table
Warranty (days)	Integer	Optional — expiry date calculated and printed on invoice
Quantity	Integer	Auto-updated on sale or purchase via Rust service

 *IMEI is validated in Rust using the Luhn algorithm before any DB write. A duplicate IMEI returns an Err() from the command — the frontend shows the error. Staff can also search sales by IMEI for warranty claims.*

6.3.2 Accessories

Field	Type	Rule
Product Name	Text	Required
Category	Dropdown	Required — from categories table
Brand	Dropdown	Required
Quantity	Integer	Auto-updated
Purchase Price	Decimal	Required
Sale Price	Decimal	Required

6.4 Sales Management Module

Feature	Detail
Create Sale	Add products to cart via React UI — invoke("create_sale") sends to Rust
Customer Linking	Link to existing customer or create inline — Rust handles atomically
Discount	Flat or percentage discount — calculated in Rust service, not frontend
Payment Types	Cash / Credit / Partial — all recorded accurately with balance computed in Rust
Stock Update	Quantity decremented in same DB transaction as the sale — no inconsistency
Invoice	PDF generated immediately by Rust after sale confirmation
Sale Return	Admin reverses a sale — Rust restores stock and voids invoice in one transaction

6.5 Invoice Management

6.5.1 PDF Generation in Rust

Invoices are generated as PDFs in the Rust backend using the printpdf crate — no third-party service or internet required.

- Shop logo, name, address, contact number
- Invoice number: auto-incremented formatted string (INV-2026-00001)
- Customer name, phone, address
- Product details: brand, model, color, storage, IMEI
- Warranty period and calculated expiry date
- Itemized list with discount and total
- Payment method and remaining balance for partial payments
- Custom footer text from settings

6.5.2 Printer Support

Printer Type	Format	Rust Crate / Approach
A4 Laser / Inkjet	Full PDF invoice — standard A4 layout	printpdf crate → OS print dialog via Tauri shell
Thermal 80mm	Receipt format — compact layout	escpos-rs crate → raw ESC/POS bytes to printer port
Thermal 58mm	Mini receipt — essential info only	escpos-rs crate → raw ESC/POS bytes to printer port

6.6 Customer Management Module

Feature	Detail
Customer Record	Name, phone, address, CNIC (optional)
Purchase History	All sales linked to this customer — loaded via Rust query
Payment History	All payments received — amounts, dates, methods
Remaining Balance	Computed in Rust from sales minus payments — always accurate
Search	Search by name or phone — Rust SQL query with LIKE pattern
Statement PDF	Printable customer account statement — generated by Rust/printpdf

6.7 Credit Management Module

- Credit sale stores total, paid amount, and remaining balance in one Rust transaction
- Record installment — invoke("add_payment") updates balance atomically
- Full payment history per credit record
- Overdue alerts — Rust compares current date to sale date, flags after configurable days
- Customer credit statement — PDF generated by Rust backend
- Total dues summary loaded on dashboard startup

6.8 Supplier Management Module

Feature	Detail
Supplier Profile	Name, contact, address, business name
Purchase History	All stock bought from this supplier
Payment Records	Payments made — what is still owed
Balance Summary	Total outstanding amount to supplier — computed in Rust

6.9 Purchase Management Module

- Record new purchase — select supplier, products, quantity, unit cost
- Stock incremented in same DB transaction as purchase record — atomic
- Purchase cost stored per item for profit calculations
- Purchase history filterable by supplier and date range

6.10 Expense Management Module

Category	Examples
Fixed	Shop rent, internet, fixed staff salary
Variable	Electricity, water, maintenance
Staff Costs	Daily wages, bonuses
Transport	Delivery, fuel
Other	Miscellaneous with free-text notes

6.11 Reports Module

Report	Description	Output
Daily Sales	All transactions for a selected date	PDF / Print
Monthly Sales	Day-by-day totals for a selected month	PDF / Print
Yearly Sales	Month-by-month totals for a year	PDF / Print
Profit Report	Revenue minus purchase cost minus expenses	PDF / Print
Product Profit	Profit per category or individual product	PDF / Print
Expense Report	Expenses by category and date range	PDF / Print
Stock Report	Current inventory with quantity and total value	PDF / Print

Report	Description	Output
Low Stock Alert	Products below configurable minimum quantity	PDF / Print
Customer Dues	All outstanding customer balances	PDF / Print
Supplier Dues	All outstanding supplier payments	PDF / Print

6.12 Settings Module

Group	Options
Shop Settings	Name, logo upload, address, phone, tagline
Invoice Settings	Number prefix, footer text, default printer, logo on/off
User Management	Create / edit / deactivate users, assign roles, reset passwords
Product Settings	Manage categories, brands, product types
Backup Settings	Backup folder path, auto-backup schedule (daily/weekly)
Stock Alerts	Minimum quantity threshold — triggers low-stock warning on dashboard
License Info	View license type, device fingerprint hash, and expiry date

7. License Management System

7.1 Overview

The license system is fully implemented in Rust. It works 100% offline — no internet required for activation or validation. Each license key is cryptographically tied to the hardware it is activated on.

7.2 Hardware Fingerprint (Rust)

```
// Rust — collect hardware IDs let cpu_id = get_cpu_id(); // WMIC or /proc/cpuinfo let disk_id =  
get_disk_serial(); // First disk serial number let mac_addr = get_mac_address(); // First non-loopback MAC //  
Combine and hash let fingerprint = sha256(cpu_id + disk_id + mac_addr); // License payload (AES-256 encrypted) let  
payload = LicensePayload { shop_name: "Noor Mobile Shop".to_string(), license_type:  
LicenseType::Professional, expiry_date: NaiveDate::from_ymd(2027, 8, 26), fingerprint: fingerprint.clone(), }; let  
license_key = aes256_encrypt(serialize(payload), SECRET_KEY);
```

7.3 Validation Flow

- On first run: user enters license key → Rust decrypts → checks fingerprint matches current hardware
- On every startup: Rust silently re-validates key against hardware fingerprint
- Mismatch: returns Err("License not valid on this device") → access denied
- Expiry: Rust compares expiry_date to today → shows renewal prompt after expiry
- Grace period: 7 days read-only access after expiry before full lockout
- License stored in AES-256 encrypted file in AppData — cannot be manually edited

7.4 License Tiers

License	Max Users	Features	Validity
Basic	1	Core sales, inventory, invoicing	1 year
Professional	Up to 5	All modules including credit, reports, multi-user	1 year
Enterprise	Unlimited	All modules, multi-branch ready, priority support	1 year

8. Security Requirements

Layer	Implementation	Standard
Database Encryption	SQLite + SQLCipher — full AES-256 file encryption	AES-256
Password Hashing	Argon2id via argon2 Rust crate — memory-hard, GPU-resistant	Argon2id
Session Auth	Signed JWT generated in Rust — stored in memory only	JWT / HS256
Backup Encryption	AES-256 encrypted .bkp archive (aes-gcm Rust crate)	AES-256-GCM
License Protection	Hardware fingerprint + AES-256 encrypted key file	SHA-256 + AES
IPC Security	All Tauri commands validate JWT token before executing	Token Auth
Permission Checks	Role validated in Rust service layer — not just frontend	RBAC
Activity Audit	Immutable log table — every action with user + timestamp	Audit Trail

⚠️ *Argon2id is preferred over bcrypt for password hashing in 2024+ because it is resistant to GPU and ASIC attacks. The argon2 crate is the standard Rust implementation.*

9. Technology Stack

9.1 Complete Stack

Layer	Technology	Purpose
Desktop Framework	Tauri 2.0	Desktop app shell — bundles Rust backend + WebView frontend
Backend Language	Rust (2024 edition)	Business logic, database access, PDF gen, license system
Frontend Language	TypeScript + React 18	UI rendering — runs inside Tauri's WebView2 window
UI Styling	Tailwind CSS	Utility-first CSS — fast, consistent, customizable
Frontend State	Zustand	Lightweight React state management
Charts	Recharts	Sales trend and profit charts on dashboard
Database	SQLite	Zero-config, serverless, single-file local database
DB Encryption	SQLCipher	AES-256 full-database encryption via rusqlite-cipher
DB Access	rusqlite + sqlx	rusqlite for direct queries; sqlx for async where needed
PDF Generation	printpdf (Rust crate)	Invoice and report PDF generation — no internet
Thermal Print	escpos-rs (Rust crate)	ESC/POS command generation for 58mm/80mm printers
Password Hashing	argon2 (Rust crate)	Argon2id password hashing — memory-hard
Encryption Utils	aes-gcm (Rust crate)	AES-256-GCM for backup and license file encryption
Hardware ID	machine-uid / sysinfo crates	CPU, disk, MAC address for license fingerprint
Logging	tracing + tracing-subscriber	Structured daily rotating log files
Installer	Tauri bundler (.msi / .exe)	Built-in Tauri build creates installer automatically
Build Tool	Vite (frontend) + Cargo (Rust)	Fast frontend bundler + Rust package manager

9.2 Why Tauri Over Electron

Criterion	Tauri + Rust	Electron + Node.js
Installer Size	~5–15 MB (uses system WebView)	~100–200 MB (bundles Chromium)
RAM Usage	Low — Rust has no garbage collector	High — V8 + Chromium overhead

Criterion	Tauri + Rust	Electron + Node.js
Performance	Native Rust speed for backend	JavaScript — slower for CPU tasks
Security	Rust memory safety — no use-after-free	Node.js — larger attack surface
PDF / Print	Rust crates — runs natively	Node.js libraries — adequate
Learning Curve	Rust is steep — longer to learn	Easier for JS developers
Ecosystem	Growing fast — Tauri 2.0 stable	Very mature — large ecosystem
Verdict	✓ Better choice for this project	✗ Heavier than needed

9.3 Why SQLite is the Right Database

Criterion	SQLite	SQL Server Express	PostgreSQL
Setup	None — zero config	Service installer	Service installer
Internet	Never needed	Never	Never
Portability	Single .db file	Complex files	Complex files
Encryption	SQLCipher — AES-256	TDE — paid feature	pgcrypto — complex
Rust Support	rusqlite + sqlx — great	tiberius crate	sqlx — great
Backup	Copy one file	Complex procedures	pg_dump required
Verdict	✓ Perfect choice	✗ Over-engineered	✗ Over-engineered

10. Database Design

10.1 Core Tables

Table	Key Fields	Purpose
users	id, username, password_hash (Argon2id), role_id, is_active	Staff accounts
roles	id, role_name, permissions (JSON text)	Role definitions
products	id, product_type, brand, model, imei (UNIQUE), purchase_price, sale_price, qty, supplier_id	All inventory
categories	id, name, product_type	Product categories
brands	id, name	Product brands
customers	id, name, phone (UNIQUE), address, total_balance	Customer records
sales	id, customer_id, user_id, total_amt, paid_amt, discount, date, payment_type	Sale header
sale_items	id, sale_id, product_id, qty, unit_price, discount	Line items per sale
purchases	id, supplier_id, total_cost, date, notes	Stock purchases
purchase_items	id, purchase_id, product_id, qty, unit_cost	Items per purchase
suppliers	id, name, phone, address, total_balance	Supplier records
payments	id, entity_id, entity_type (customer/supplier), amount, date, method	All payments
expenses	id, category, amount, date, description	Business expenses
activity_log	id, user_id, action, table_name, record_id, timestamp	Immutable audit trail
settings	key TEXT PRIMARY KEY, value TEXT	All app configuration
license	license_key, fingerprint_hash, license_type, expiry_date, activated_at	License data

10.2 Key Design Rules

- All tables use INTEGER PRIMARY KEY (SQLite auto-increment rowid alias)
- IMEI column has UNIQUE constraint at DB level — duplicate rejected even if Rust check is bypassed
- All monetary values stored as INTEGER cents (e.g., 15000 = Rs 150.00) — avoids floating-point precision issues
- Soft delete pattern — records have is_deleted BOOLEAN, never physically removed (preserves history)
- activity_log is append-only — no UPDATE or DELETE allowed on this table (enforced in Rust)
- Foreign key constraints enabled via PRAGMA foreign_keys = ON on every connection

11. Backup and Restore

11.1 Backup File Structure

```
NoorMobileShop_Backup_2026-08-26.bkp (AES-256-GCM Encrypted Archive — created by Rust aes-gcm crate)
├── shop_database.db    (the encrypted SQLite file)  ├── settings_export.json (all settings key-value pairs)
├── shop_logo.png      (shop logo)    └── backup_meta.json  (date, app version, shop name, record counts)
```

11.2 Backup Features

Feature	Detail
Manual Backup	Admin triggers backup via UI → invoke("create_backup") → Rust creates .bkp file
Auto Backup	Tauri background task — configurable daily/weekly schedule
Encryption	AES-256-GCM — file is unreadable without the app decrypting it
Restore	Select .bkp → invoke("restore_backup") → Rust validates metadata, decrypts, restores
Version Check	backup_meta.json stores app version — Rust rejects incompatible backup versions
Naming Format	ShopName_Backup_YYYY-MM-DD.bkp — easy to sort and identify

12. Error Handling and Logging

12.1 Rust Error Handling

All Tauri commands return `Result<T, AppError>`. `AppError` is a custom Rust enum with variants for every error type. The frontend receives a structured JSON error — never a panic or raw exception.

```
[derive(Debug, Serialize)] enum AppError { DatabaseError(String), DuplicateImei(String), LicenseInvalid, PermissionDenied, ValidationError(String), PrintError(String), }
```

12.2 Logging with tracing

Level	When Used	Example
INFO	Normal events	User logged in, sale created, backup completed
WARN	Non-critical issues	Low stock reached, license expiring in 7 days
ERROR	Caught Rust errors	DB write failed, print error, file not found
DEBUG	Development only	SQL queries, IPC payload details (disabled in release build)

- Log files: `AppData\MobileShopPro\logs\app-YYYY-MM-DD.log`
- Daily rolling files — auto-delete after 30 days
- Release builds: INFO level and above only — no debug noise for customers
- Error dialog shows reference code matching the log entry — easy remote support

13. Non-Functional Requirements

Category	Requirement
Performance	App startup under 2 seconds; search results under 300ms (Rust + SQLite is very fast)
Reliability	Zero internet dependency; SQLite WAL mode prevents data loss on power cut
Usability	New staff can complete a sale within 10 minutes without training documentation
Security	All sensitive data AES-256 encrypted at rest; Argon2id password hashing
Scalability	SQLite comfortably handles 100,000+ product records for a single shop
Maintainability	Database migrations via versioned SQL scripts applied on startup
Compatibility	Windows 10 and Windows 11 (x64); WebView2 auto-installed if missing
Installer	Single .msi or .exe from Tauri bundler; installs in under 60 seconds
Bundle Size	Target under 20 MB installer — no bundled Chromium (uses system WebView2)

14. Development Build Roadmap

Phase	Focus	Deliverable
Phase 1	Project Setup	Tauri 2.0 + React + Rust workspace; Cargo.toml; Vite config; SQLite connection with SQLCipher
Phase 2	Database Schema	SQL migration files; all tables created on first run; rusqlite repository base struct
Phase 3	Authentication	Argon2id login; JWT session; role/permission system; activity log; all validated in Rust
Phase 4	Inventory	Product CRUD (mobiles + accessories); IMEI Luhn validation; low-stock logic in Rust
Phase 5	Sales Core	Sale creation; stock decrement; printpdf invoice; thermal print via escpos-rs
Phase 6	Customers & Credit	Customer records; credit sales; installment payment tracking; balance computation
Phase 7	Suppliers	Supplier records; purchase recording; stock increment; payment tracking
Phase 8	Finance	Expense tracking; profit calculation (revenue - cost - expenses); all 10 reports as PDFs
Phase 9	Dashboard	Real-time KPI cards; monthly chart via Recharts; low-stock and dues alerts
Phase 10	License System	Hardware fingerprint in Rust; AES-256 key validation; expiry and grace period logic
Phase 11	Backup & Settings	AES-GCM backup/restore; all settings panels; logo upload; printer selection
Phase 12	QA & Polish	Error handling review; tracing log verification; UI polish; performance profiling
Phase 13	Deployment	Tauri bundler → .msi and .exe installer; code signing (optional); release build

15. Future Enhancements

Version	Planned Features
v2.0	Multi-branch support; repair/service job management; barcode scanner via Tauri serial port plugin; advanced accounting module
v3.0	Optional cloud sync with end-to-end encryption; web dashboard for remote monitoring; customer WhatsApp notifications via Twilio
v4.0	Mobile owner app (Tauri on Android/iOS); AI sales forecasting; customer self-service payment records

16. Conclusion

Mobile Shop Manager Pro is built on a modern, production-grade Tauri + Rust + SQLite stack that delivers a small installer (under 20 MB), native-speed performance, and AES-256 encrypted local storage — with zero internet dependency.

Rust enforces business logic, security, and data integrity at the backend layer. The React + TypeScript frontend provides a fast, modern UI. SQLite with SQLCipher gives a zero-configuration encrypted database in a single portable file.

The hardware-bound license system, Argon2id password hashing, AES-256-GCM encrypted backups, and a full activity audit trail make this a commercial-grade product suitable for real business deployment from day one.

| Version 1.0 | August 2026 | Confidential | Author: Noor Ullah | Stack: Tauri + Rust + SQLite